

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC

CÔNG NGHỆ THÔNG TIN

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT
NAM

Độc Lập - Tự Do - Hạnh Phúc

ĐỀ CƯƠNG CHI TIẾT

TÊN ĐỀ TÀI: NGHIÊN CỨU HỆ THỐNG ĐẶT PHÒNG KHÁCH SẠN TRỰC TUYẾN THEO KIẾN TRÚC MICROSERVICES VÀ MULTI-AGENT AI

TÊN ĐỀ TÀI (tiếng Anh): RESEARCH HOTEL BOOKING PLATFORM WITH MICROSERVICES ARCHITECTURE AND MULTI-AGENT AI SYSTEM.

Cán bộ hướng dẫn: Đỗ Minh Tiến

Thời gian thực hiện: Từ ngày 20/07/2026 đến ngày 18/09/2026

Sinh viên thực hiện:

**Giang Hải Chương –
25410179**

Giới thiệu đề tài :

Sự phát triển mạnh mẽ của ngành du lịch và dịch vụ lưu trú tại Việt Nam đang đặt ra yêu cầu cấp thiết về các nền tảng đặt phòng trực tuyến hiệu quả, đáng tin cậy và thông minh. Tuy nhiên, các hệ thống đặt phòng truyền thống hiện nay thường bộc lộ những hạn chế nghiêm trọng về khả năng mở rộng, tính sẵn sàng cao, và trải nghiệm người dùng, đặc biệt khi lưu lượng truy cập tăng đột biến vào các mùa cao điểm. Việc xử lý đồng thời lượng lớn giao dịch thường dẫn đến các vấn đề về thất cổ chai hiệu năng và rủi ro bất đồng bộ dữ liệu.

Để giải quyết những thách thức trên, đề tài "**NGHIÊN CỨU HỆ THỐNG ĐẶT PHÒNG KHÁCH SẠN TRỰC TUYẾN THEO KIẾN TRÚC MICROSERVICES VÀ MULTI AGENT AI**" được thực hiện nhằm xây dựng một nền tảng hiện đại với các đặc điểm trọng tâm sau:

- Hệ thống được xây dựng trên nền tảng kiến trúc Microservices với Java Spring Boot, chia nhỏ nghiệp vụ thành các service độc lập (Auth, Hotel, Booking, Payment, Notification), đảm bảo khả năng triển khai, mở rộng và bảo trì độc lập từng thành phần.
- Tích hợp Keycloak làm Identity Provider trung tâm để quản lý xác thực và phân quyền, đảm bảo bảo mật theo chuẩn OAuth2/OpenID Connect mà không cần tự quản lý JWT thủ công.
- Áp dụng các pattern thiết kế hiện đại: Outbox Pattern để đảm bảo tính nhất quán dữ liệu giữa service, Idempotent Consumer để tránh xử lý trùng lặp message từ Kafka, và

Distributed Locking với Redisson để kiểm soát race condition khi cập nhật inventory.

- Nâng cấp lên hệ thống Multi-Agent AI sử dụng Spring AI tích hợp Claude API, cho phép người dùng tương tác tự nhiên qua ngôn ngữ thông thường để tìm kiếm, đặt phòng và được hỗ trợ bởi các agent chuyên biệt.

Mục tiêu đề tài:

Nghiên cứu và phát triển một nền tảng đặt phòng khách sạn trực tuyến hoàn chỉnh theo kiến trúc Microservices, tích hợp hệ thống Multi-Agent AI nhằm tự động hóa và cá nhân hóa trải nghiệm đặt phòng, đảm bảo tính sẵn sàng cao, khả năng mở rộng linh hoạt và bảo mật theo chuẩn công nghiệp.

- ***Xây dựng hạ tầng Microservices:***

- Thiết kế và triển khai 7 microservice độc lập: API Gateway, Auth Service (Keycloak), Hotel Service, Booking Service, Payment Service, Notification Service và Service Registry (Eureka).
- Cấu hình Spring Cloud Config Server tập trung hóa cấu hình, Eureka Service Discovery để các service tự tìm thấy nhau, và API Gateway với JWT filter để bảo vệ toàn bộ hệ thống.
- Triển khai Docker Compose cho môi trường local và CI/CD pipeline với Gitlab Runner.

- ***Phát triển nghiệp vụ cốt lõi:***

- Hotel Service: Quản lý khách sạn, loại phòng, lịch phòng trống và hệ thống đánh giá (review).
- Booking Service: Luồng đặt phòng hoàn chỉnh với state machine (PENDING → CONFIRMED → CANCELLED → COMPLETED), tích hợp Web Client gọi Hotel Service.
- Payment Service: Tích hợp cổng thanh toán VNPay/Stripe, xử lý webhook callback và publish event qua Kafka.
- Notification Service: Consume event từ Kafka và gửi email xác nhận với template Thymeleaf.

- ***Đảm bảo tính nhất quán và an toàn dữ liệu:***

- Triển khai Outbox Pattern để đảm bảo dữ liệu được publish lên message broker ngay cả khi service gặp sự cố.
- Áp dụng Idempotent Consumer với process_id để tránh xử lý trùng lặp message.
- Sử dụng Distributed Lock (Redisson + Redis) để kiểm soát race condition khi giảm số lượng phòng.

- ***Xây dựng hệ thống Multi-Agent AI:***

- Phát triển Tool Layer wrap các REST API của microservices thành tool cho agent gọi.
- Xây dựng Orchestrator Agent nhận ngôn ngữ tự nhiên, phân tích intent và điều phối 4 sub-agent: Search, Booking và Support Agent.
- Tích hợp long-term memory với pgvector và RAG để cá nhân hóa gợi ý dựa trên lịch sử đặt phòng.

Phạm vi nghiên cứu:

- **Phạm vi kỹ thuật:**

- Ngôn ngữ và framework: Java 21, Spring Boot 3.5.9, Spring Cloud 2025.
- Message broker: Kafka.
- Database: PostgreSQL (dữ liệu nghiệp vụ), Redis (cache, session, distributed lock), pgvector (agent memory).
- Authentication: Keycloak 22 với OAuth2/OpenID Connect.
- AI/LLM: Claude API (Anthropic) hoặc Gemini API tích hợp qua Spring AI.
- Infrastructure: Docker Compose, Gitlab Runner CI/CD.

- **Phạm vi nghiệp vụ:**

- Quản lý khách sạn: CRUD hotel, phòng, loại phòng, tiện ích, chính sách hủy phòng.
- Đặt phòng: Tìm kiếm theo địa điểm/ngày, kiểm tra phòng trống, đặt phòng, thanh toán, hủy đặt phòng.
- Người dùng: Đăng ký, đăng nhập, quản lý profile, lịch sử đặt phòng.
- Thông báo: Email xác nhận đặt phòng, email hủy đặt phòng, cảnh báo.
- Multi-Agent: Tìm kiếm bằng ngôn ngữ tự nhiên, đặt phòng qua chat, hỗ trợ hủy đặt phòng.

Đối tượng sử dụng:

- **Khách hàng (Guest):** Người dùng cuối muốn tìm kiếm và đặt phòng khách sạn trực tuyến. Có thể tương tác qua giao diện web truyền thống hoặc qua chatbot AI với ngôn ngữ tự nhiên.
- **Chủ khách sạn (Hotel Owner):** Người quản lý danh sách khách sạn, phòng, giá phòng và xem báo cáo đặt phòng. Có quyền quản lý inventory và thiết lập chính sách hủy phòng.
- **Quản trị hệ thống (Admin):** Người quản lý toàn bộ hệ thống, phê duyệt khách sạn mới, quản lý người dùng và giám sát hoạt động hệ thống.
- **Hệ thống AI Agent:** Các agent tự động hóa quy trình đặt phòng, hỗ trợ khách hàng 24/7, gợi ý phòng theo sở thích người dùng.

Nội dung và phương pháp thực hiện:

- **Nội dung 1: Xây dựng hạ tầng và Infrastructure**

Mục tiêu: Thiết lập nền móng kỹ thuật cho toàn bộ hệ thống, đảm bảo các service có thể giao tiếp với nhau và được quản lý tập trung.

- **Phương pháp:**

- Tổ chức Mono-repo với Maven Multi-module: Toàn bộ source code của 10 module(common-utils,common-dto,service-registry,config-server,api-gateway,auth-service,hotel-service,booking-service,payment-service,notification-service) được quản lý trong một repository duy nhất. Pom cha quản lý version tập trung, các pom con kế thừa và chỉ khai báo dependency đặc thù.
- Service Registry (Eureka): Các service tự đăng ký địa chỉ khi khởi động, API Gateway và Web Client tự động tìm service qua tên logic mà không

cần hardcode URL.

- Config Server (Spring Cloud Config): Tập trung hóa file cấu hình (application.yml) cho tất cả service, hỗ trợ thay đổi cấu hình không cần restart service.
- API Gateway (Spring Cloud Gateway): Điểm vào duy nhất của hệ thống, thực hiện JWT validation, routing và rate limiting trước khi forward request đến service tương ứng.
- Docker Compose: Orchestrate toàn bộ infrastructure local bao gồm PostgreSQL, Redis, Kafka, ELK, Keycloak, bằng một lệnh docker-compose up.
- Gitlab Runner CI/CD: Tự động build và Docker build cho tất cả module khi có push lên main branch.

- **Nội dung 2: Xây dựng Auth Service với Keycloak**

Mục tiêu: Thiết lập hệ thống xác thực và phân quyền tập trung, đảm bảo bảo mật theo chuẩn OAuth2/OpenID Connect mà không cần tự quản lý JWT thủ công.

- **Phương pháp:**

- Keycloak 22 làm Identity Provider: Quản lý toàn bộ vòng đời user (tạo, xóa, kích hoạt, đặt lại mật khẩu), cấp phát JWT token và refresh token, quản lý role (USER, OWNER, ADMIN).
 - Keycloak Admin Client: Auth Service gọi Keycloak Admin REST API để tạo user khi đăng ký, gán role và đồng bộ thông tin. Không lưu mật khẩu trong database của hệ thống.
 - UserProfile Entity: Lưu thông tin bổ sung (phone, avatar, dateOfBirth, gender, nationality) mà Keycloak không quản lý, liên kết với Keycloak qua keycloakId (UUID do Keycloak cấp).
 - OAuth2-Resource-Server: Các service khác cấu hình spring-boot-starter-oauth2-resource-server để tự động validate JWT token qua JWKS endpoint của Keycloak, không cần parse token thủ công.
 - Cross-field Validation: Áp dụng custom annotation và ConstraintValidator để validate business rule phức tạp tại tầng DTO.

- **Nội dung 3: Xây dựng Hotel Service**

Mục tiêu: Phát triển core domain quản lý khách sạn đảm bảo tính nhất quán dữ liệu và hiệu năng truy vấn.

- **Phương pháp:**

- Domain Model: Xây dựng các JPA entity với BaseEntity (dùng @SuperBuilder + @MappedSuperclass) chứa audit fields (createdBy, createDate, lastModifiedBy, lastModifiedDate) và soft delete (isDeleted). Các entity chính: Hotel, RoomType, Amenity, HotelReview với các embedded object: Address (bao gồm tọa độ lat/long), CancellationPolicy.
 - Hotel Management (CRUD hotel, amenities).
 - Outbox Pattern: Sau mỗi thao tác quan trọng (tạo hotel, cập nhật

availability), hệ thống lưu event vào bảng outbox trong cùng transaction. Một scheduler định kỳ đọc outbox và publish lên Kafka, đảm bảo không mất event kể cả khi service bị restart.

- Idempotent Consumer + Distributed Lock: Khi consume message booking từ Kafka, hệ thống check processId và dùng Redisson MultiLock để lock nhiều RoomType cùng lúc theo thứ tự sorted IDs tránh deadlock.
- MapStruct Mapper: Sử dụng @Mapper với NullValuePropertyMappingStrategy.IGNORE để tự động map Entity ↔ DTO, hỗ trợ partial update (chỉ cập nhật field không null).

- **Nội dung 4: Xây dựng Booking Service cùng với Payment Service và Notification Service.**

Mục tiêu: Triển khai luồng đặt phòng hoàn chỉnh với state machine, tích hợp thanh toán cùng với thông báo và đảm bảo tính nhất quán dữ liệu qua message-driven architecture.

- **Phương pháp:**

- Booking State Machine: Booking entity có trạng thái PENDING → CONFIRMED → CANCELLED / COMPLETED. Mỗi chuyển trạng thái đều validate điều kiện và publish event tương ứng lên Kafka.
- Web Client với Load Balancing: Booking Service gọi Hotel Service qua Web Client để check availability và hold room. Tích hợp Spring Cloud LoadBalancer để tự động chọn instance nếu Hotel Service chạy nhiều bản.
- Cross-field Validation tại DTO: BookingRequest áp dụng @CheckOutAfterCheckIn, @FutureOrPresent cho checkInDate, @Future cho checkOutDate, @Min/@Max cho quantity. Validate toàn bộ trước khi save bất kỳ entity nào vào database.
- @TransactionalEventListener: Kafka send sau khi transaction commit thành công (AFTER_COMMIT phase), tránh trường hợp message đã gửi nhưng transaction bị rollback.
- Payment Service: Consume event booking.confirmed từ Kafka, thực hiện charge qua Stripe/VNPay, xử lý webhook callback và publish payment.completed event. Jackson ObjectMapper được cấu hình với JavaTimeModule (tắt WRITE_DATES_AS_TIMESTAMPS) và custom BigDecimalSerializer để tránh scientific notation (6.5E+6 → 6500000).
- Notification Service: Consume event booking.confirmed.paid or failed từ Kafka thực hiện gửi mail or sms cho người dùng và lưu vào database.

- **Nội dung 5: Xây dựng Multi-Agent AI System**

Mục tiêu: Nâng cấp hệ thống từ REST API thuần túy lên Multi-Agent AI System, cho phép người dùng tương tác bằng ngôn ngữ tự nhiên và tự động hóa toàn bộ luồng đặt phòng.

- **Phương pháp:**

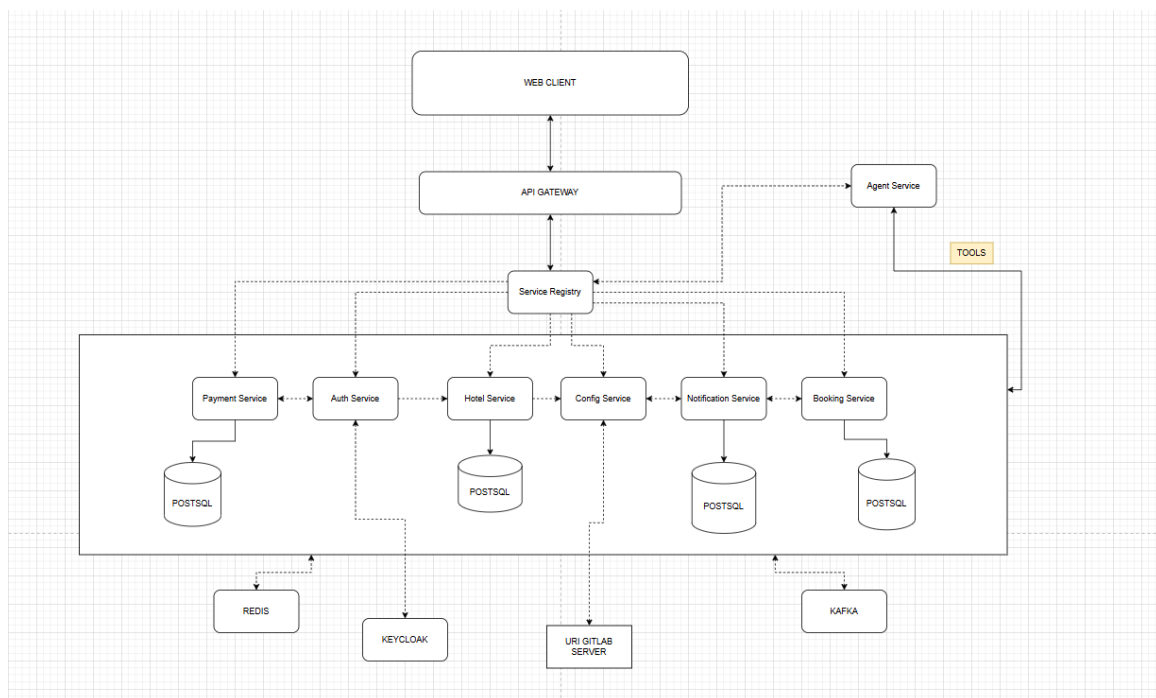
- Tool Layer (Spring AI): Wrap toàn bộ REST API của microservices thành @Tool functions với schema rõ ràng để LLM biết khi nào gọi và truyền tham số gì. Các tool chính: search_hotels, check_availability,

create_booking, calculate_price, cancel_booking, send_notification.

- Orchestrator Agent: Nhận chat input từ user, sử dụng Claude API qua Spring AI ChatClient để phân tích intent (search/book/cancel/support) và điều phối 4 sub-agent chuyên biệt. Lưu conversation history ngắn hạn trong Redis.
- Search Agent: Xử lý natural language query, gọi tool searchHotels và format lại kết quả cho thân thiện với người dùng.
- Booking Agent: Thực hiện human-in-the-loop trước khi tạo booking và thanh toán. Quản lý booking state machine qua Redis: SEARCHING → SELECTED → CONFIRMING → BOOKED.
- Long-term Memory với pgvector: Lưu lịch sử đặt phòng và preference của user dưới dạng vector embedding. Sử dụng RAG để agent gợi ý khách sạn phù hợp dựa trên hành vi lịch sử.

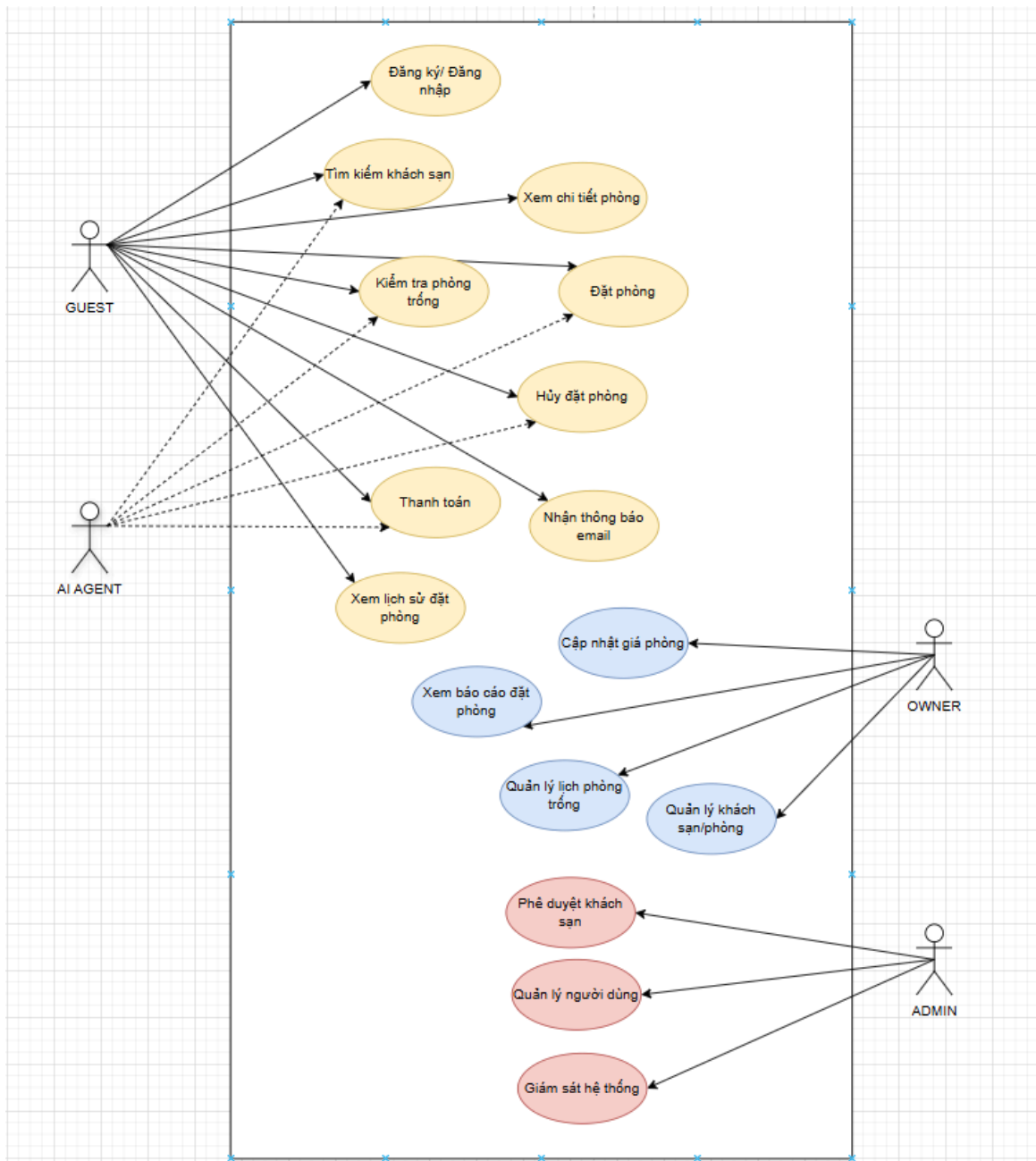
Kiến trúc hệ thống:

- *Sơ đồ kiến trúc tổng thể:*



Hình 6.1

- *Sơ đồ usecase:*



Hình 6.2

• **Bảo mật:**

- Keycloak: Cấp phát và quản lý JWT token, hỗ trợ Social Login (Google, Facebook) nếu cần mở rộng.
- API Gateway: Validate JWT token trước khi forward, extract user info và inject vào header.
- Service Level: Mỗi service cấu hình OAuth2 Resource Server để verify token, dùng @AuthenticationPrincipal Jwt để lấy keycloakId từ sub claim.

Kết quả mong đợi:

• **Core System:**

- Hệ thống Microservices hoàn chỉnh với 7 service chạy độc lập, giao tiếp qua Kafka và REST (hoặc có thể change sang GRPC để optimize), có thể scale từng service riêng lẻ.
- Tích hợp Keycloak đảm bảo xác thực bảo mật, hỗ trợ SSO và phân quyền role-based.
- Outbox Pattern + Idempotent Consumer + Distributed Lock đảm bảo tính nhất quán dữ liệu trong môi trường phân tán.
- CI/CD pipeline tự động build và push docker hub khi có thay đổi code.
- **Business:**
 - Luồng đặt phòng end-to-end hoàn chỉnh: tìm kiếm → chọn phòng → thanh toán → xác nhận email.
 - Quản lý khách sạn đầy đủ: CRUD hotel/room, availability calendar, dynamic pricing, review system.
 - Hỗ trợ hủy đặt phòng tự động.
- **Multi - Agent System:**
 - Người dùng có thể đặt phòng bằng ngôn ngữ tự nhiên tiếng Việt qua chat interface.
 - Agent nhớ lịch sử đặt phòng và preference, gợi ý khách sạn phù hợp cá nhân hóa.
 - Support Agent xử lý 80% yêu cầu hỗ trợ tự động, chỉ escalate khi thực sự cần con người.
- **Scalability:**
 - Kiến trúc cho phép thêm service mới (Review Service, Analytics Service) mà không ảnh hưởng service hiện tại.
 - Multi-Agent có thể thêm agent mới (Tour Agent, Transfer Agent) bằng cách implement tool mới.

Kế hoạch thực hiện:

- **Phase 1: Thiết lập nền móng kiến trúc hệ thống phân tán phân tầng (20/7 -25/7)**
 - Tạo Maven Multi-module project với pom cha quản lý version tập trung
 - Định nghĩa common: chứa các DTO dùng chung, BaseEntity, exception handler, response wrapper
 - Khởi tạo Spring Boot app với dependency spring-cloud-starter-netflix-eureka-server
 - Cấu hình application.yml: tắt self-registration, đặt port 8761
 - Khởi tạo Spring Cloud Config Server, trỏ tới Git repo chứa file cấu hình
 - Tạo file application.yml cho từng service trong Git repo config
 - Cấu hình Spring Cloud Gateway với routing rules cho từng service
 - Viết JWT Filter: extract token từ header, validate với Keycloak JWKS endpoint, inject user info vào downstream header
 - Viết docker-compose.yml orchestrate: PostgreSQL, Redis, Kafka, Keycloak, Eureka, Config Server
 - Tạo Gitlab Runner workflow: build, Docker build cho tất cả module khi push lên main.
- **Phase 2: Thiết lập Identity (26/7 - 6/8)**

- Tạo Realm, Client, và 3 Role: USER, OWNER, ADMIN
- Cấu hình token expiry, refresh token, JWKS endpoint cho các service khác validate
- Cấu hình gọi API theo OIDC để login, register, getprofile, refreshtoken
- Thêm spring-boot-starter-oauth2-resource-server vào API Gateway để xác thực tập trung (validate JWT qua JWKS, extract keycloakId từ sub claim).

- **Phase 3: Xây dựng Hotel Service (7/8 - 19/8)**

- Tạo BaseEntity với @MappedSuperclass + @SuperBuilder: createdBy, createdDate, lastModifiedBy, lastModifiedDate, isDeleted (soft delete)
- Tạo Entity chính: Hotel, RoomType, Amenity, HotelReview
- Viết MapStruct Mapper với NullValuePropertyMappingStrategy.IGNORE cho partial update
- CRUD API cơ bản cho hotel
- Dùng Redisson MultiLock theo sorted RoomType IDs để tránh deadlock khi giảm inventory
- Sau thao tác CRUD, lưu event vào bảng outbox_events trong cùng transaction. Scheduler đọc & publish lên Kafka
- Kiểm tra processId trước và sau Distributed Lock để tránh xử lý trùng message

- **Phase 4: Xây dựng Booking Service (20/8 - 28/8)**

- Thiết kế Booking entity với field status: PENDING → CONFIRMED → CANCELLED / COMPLETED
- Cấu hình Web Client và Spring Cloud LoadBalancer để check availability & hold room bên Hotel Service
- CRUD API cơ bản cho booking
- Dùng @TransactionalEventListener(phase = AFTER_COMMIT): chỉ publish Kafka event sau khi transaction commit thành công
- Event payload: booking.confirmed (gửi sang Payment Service), booking.cancelled (gửi sang Notification)
- Mỗi booking event lưu Outbox trước rồi scheduler poll & publish để đảm bảo không mất event khi service restart

- **Phase 5: Xây dựng Payment và Notification Service (29/8 - 6/9)**

- Consume event booking.confirmed từ kafka và khởi tạo payment session với VNPAY hoặc Stripe
- Tạo payment URL với HMAC-SHA512 signature, redirect user đến VNPAY gateway
- Cấu hình webhook endpoint nhận callback từ VNPAY/Stripe, verify signature, cập nhật payment status
- Sau payment thành công: publish event payment.completed sang Kafka update Booking sang COMPLETED
- Consume event payment.completed và booking.cancelled từ kafka
- Thiết kế email template bằng Thymeleaf: xác nhận đặt phòng (tên KS, ngày, giá), email hủy (lý do, tiền hoàn)
- Gửi email qua Spring Mail (SMTP), cấu hình từ Config Server để dễ thay đổi provider

- **Phase 6: Xây dựng Tool Layer - Spring AI Tool (7/9 - 12/9)**

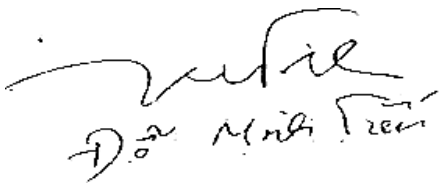
- Tạo module ai-agent-service trong mono-repo, thêm dependency spring-ai-anthropic-spring-boot-starter
- Mỗi tool là một Java method annotated @Tool(name, description) mô tả phải rõ ràng để Claude biết khi nào gọi
- Implement từng tool như search_hotels, check_availability, create_booking, cancel_booking,...
- Viết test gọi ChatClient với prompt
- **Phase 7: Xây dựng Multi-Agent AI System (13/9 - 18/9)**
 - Build Orchestrator để nhận chat input, dùng ChatClient gọi Claude API phân tích các intent như search/book/cancel/support
 - Điều phối sang đúng sub-agent theo intent, truyền context (userId, conversationId)
 - Lưu conversation history ngắn hạn trong Redis

Công nghệ sử dụng:

- Java 21
- SpringBoot 3.5.9, SpringCloud 2025.0.1
- Keycloak 22.0.5
- PostgreSQL 17
- Redis 7.x & Redisson 3.52.0
- Kafka 2.8.1
- Spring Data JPA 3.x & Hibernate 3.x
- MapStruct 1.5.5
- Spring AI 1.x
- Claude API (Anthropic) claude-sonnet-4-6
- Docker 28.2.2 + Docker Compose 28.2.2
- Thymeleaf 3.x

Tài liệu tham khảo:

- Spring Boot Documentation – <https://docs.spring.io/spring-boot/>
- Spring Cloud Documentation – <https://spring.io/projects/spring-cloud>
- Keycloak Server Administration Guide – <https://www.keycloak.org/documentation>
- Spring AI Documentation – <https://docs.spring.io/spring-ai/>
- Anthropic Claude API Documentation – <https://docs.anthropic.com/>
- Redisson Documentation – <https://redisson.org/documentation.html>
- Richardson, C. (2018). Microservices Patterns. Manning Publications.
- Newman, S. (2021). Building Microservices (2nd ed.). O'Reilly Media.
- pgvector Documentation – <https://github.com/pgvector/pgvector>
- Kafka Documentation – <https://kafka.apache.org/documentation>

Xác nhận của CBHD (Ký tên và ghi rõ họ tên) 	TP. HCM, ngày 20 tháng 07 năm 2026 Sinh viên (Ký tên và ghi rõ họ tên) Giang Hải Chương
---	--